TECHNICAL DOCUMENTATION

# Lumen

## AI Freelance Proposal & Cover Letter Assistant

Win more freelance clients with AI — proposals, cover letters & outreach in seconds.

Next.js 15 | TypeScript 5 | PostgreSQL + Prisma | Groq · Llama 3.1 8B | JWT Auth | Tailwind CSS

Architecture · Features · Data Model · API Reference

Version 1.0 · June 2026

# 1. Overview

**Lumen** is a production-style AI SaaS that helps freelancers, developers, designers, and job seekers generate high-converting proposals, cover letters, and outreach. A user pastes a job brief, picks a target platform and tone, and the app produces tailored copy, scores it for conversion, estimates pricing and timeline, and keeps everything organised in a dashboard.

The application is a single **Next.js 15 App Router** codebase that serves both the marketing site and the authenticated product. All AI calls run server-side through a single Groq client, all data persists in PostgreSQL via Prisma, and authentication is handled with custom stateless JWT sessions stored in an HttpOnly cookie.

> **AI provider:** The project was migrated from Google Gemini to **Groq**. Every AI feature now routes through `src/lib/groq.ts`, which calls Groq's OpenAI-compatible Chat Completions API using the `llama-3.1-8b-instant` model (configurable via the `GROQ_MODEL` environment variable).

## Core capabilities

- **AI Proposal Generator** — tailors a proposal to the brief across 7 platforms (Upwork, Fiverr, LinkedIn, Freelancer, cold email, agency pitch, general).
- **Cover Letter Writer** — ATS-friendly letters that mirror job-post keywords.
- **Brief Analyzer** — categorises the project and surfaces intent, pain points, red flags, and a suggested tone (strict-JSON output).
- **Conversion Scoring** — grades copy 0–100 across relevance, specificity, conversion, voice, and length, with concrete fixes.
- **Pricing & Timeline AI** — fixed price, hourly range, total hours and calendar days, with rationale and risks.
- **Streaming Chat Copilot** — token-by-token SSE streaming with persistent sessions.
- **Inline Rewrites** — shorter, more premium, more technical, punchier opener, stronger CTA, ATS keywords.
- **Workspace** — saved proposals, cover letters, saved jobs, templates, analytics, and PDF export.

## 2. Technology Stack

Lumen is intentionally dependency-light. There is no AI SDK, no state-management library, and no heavyweight UI framework — the AI client and auth layer are hand-written and fully owned.

| Layer | Technology | Role in the system |
|---|---|---|
| Framework | Next.js 15 (App Router, RSC, Server Actions) | Routing, server components, API route handlers, middleware. |
| Language | TypeScript 5 | End-to-end static typing across client and server. |
| UI runtime | React 19 | Client components for interactive dashboard surfaces. |
| Styling | Tailwind CSS 3 + CSS variables | Utility-first styling with a custom design-token theme. |
| Components | Radix UI + shadcn-style + Framer Motion | Accessible primitives, composed components, animation. |
| Icons | Lucide React | Icon set across marketing and dashboard. |
| Database | PostgreSQL | Relational store for users and all generated artifacts. |
| ORM | Prisma 5 | Schema, typed client, migrations, seeding. |
| AI | Groq (Llama 3.1 8B Instant) REST + SSE | Text generation, strict-JSON output, and streaming chat. |
| Auth | jose (JWT) + bcryptjs | Stateless HS256 sessions; password hashing at cost 12. |
| Validation | zod | Request body parsing & input validation on every route. |
| Markdown | react-markdown + remark-gfm | Rendering generated copy and chat responses. |
| PDF export | jsPDF | Client-side export of proposals / cover letters. |
| Notifications | sonner | Toast feedback in the dashboard. |
| Packaging | Docker + docker-compose | Containerised app + Postgres for local/prod parity. |

# 3. System Architecture

Every request enters through Next.js middleware, which enforces auth on `/dashboard` routes before any page or API handler runs. Server-side route handlers call three shared libraries — the Groq client, the auth helpers, and the Prisma client — which in turn talk to the two external dependencies: the Groq API and PostgreSQL.

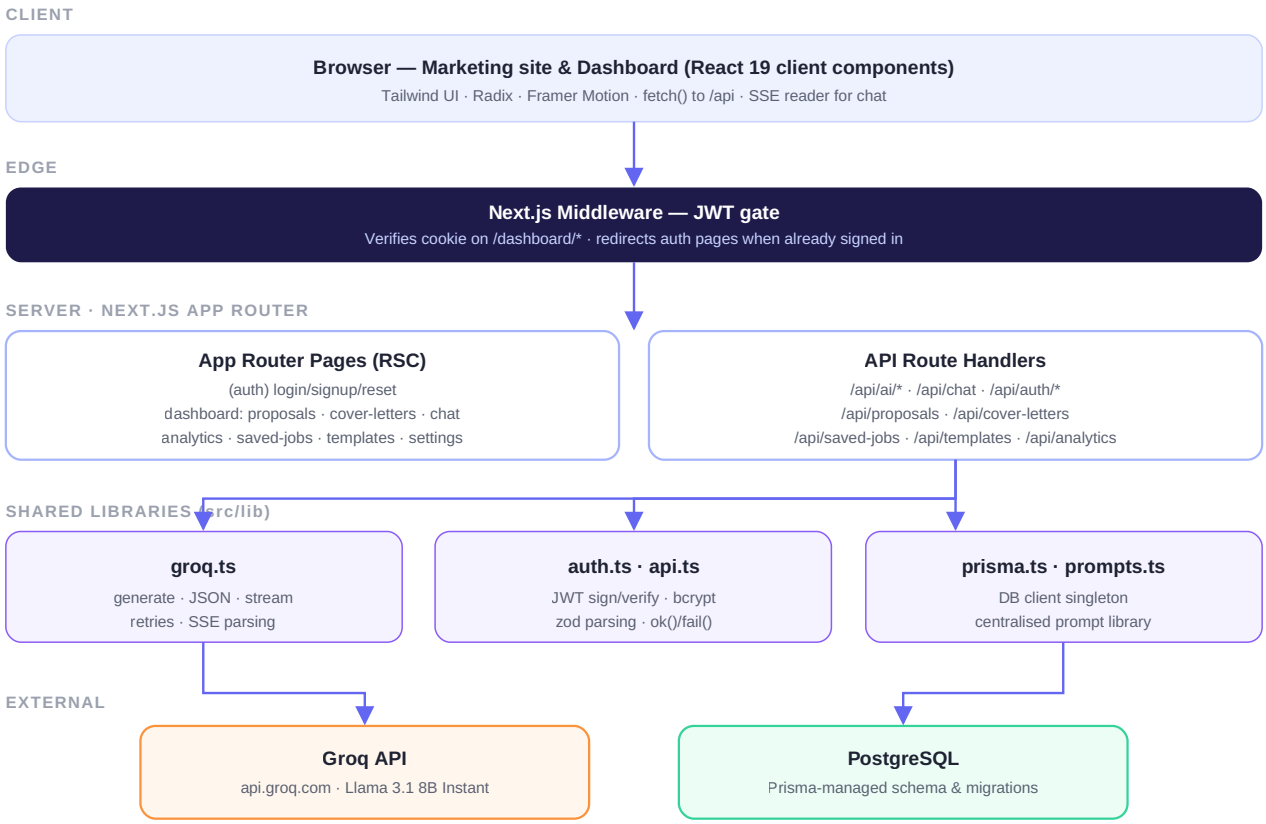


*Figure 1 — Layered request architecture, from browser through middleware and route handlers to shared libraries and external services.*

## Request lifecycle

- **Middleware** ( `src/middleware.ts` ) verifies the JWT cookie and gates `/dashboard/*` ; signed-in users are bounced away from auth pages.
- **Route handlers** call `requireAuth()` , validate the body with a zod schema via `readBody()` , then perform work and return a uniform `{ ok, data }` / `{ ok, error }` envelope.
- **AI work** is funnelled through `src/lib/groq.ts` so model, retries, and streaming live in exactly one place.
- **Persistence** goes through a single Prisma client singleton, and most generation routes also write an `AiGeneration` audit row and an `AnalyticsEvent` .

# 4. AI Layer — Groq Integration

The entire AI surface is a single dependency-free client, `src/lib/groq.ts`, that wraps Groq's OpenAI-compatible `/openai/v1/chat/completions` endpoint. Routes never call the network directly — they import one of four functions.

| Export | Purpose |
|---|---|
| `groqGenerate()` | One-shot text completion. Built-in retry with exponential back-off on HTTP 429 / 5xx. |
| `groqJSON<T>()` | Strict JSON mode (`response_format: json_object`) with defensive code-fence stripping and typed parse. |
| `groqStream()` | SSE streaming → returns a `ReadableStream` of decoded text deltas, buffered across chunk boundaries. |
| `asChat()` | Maps `{role, content}[]` chat history into the provider message format. |

> Model and tuning are environment-driven: `GROQ_API_KEY` and `GROQ_MODEL` (default `llama-3.1-8b-instant`). Per-call options cover `system`, `temperature`, `topP`, `maxOutputTokens`, `json`, and `retries`.

All prompts are centralised in `src/lib/prompts.ts` — proposal, cover-letter, rewrite, score, pricing, follow-up, and the chat system prompt — with shared anti-pattern rules that block AI-cliché openers and buzzwords to keep output human.

## AI generation flow

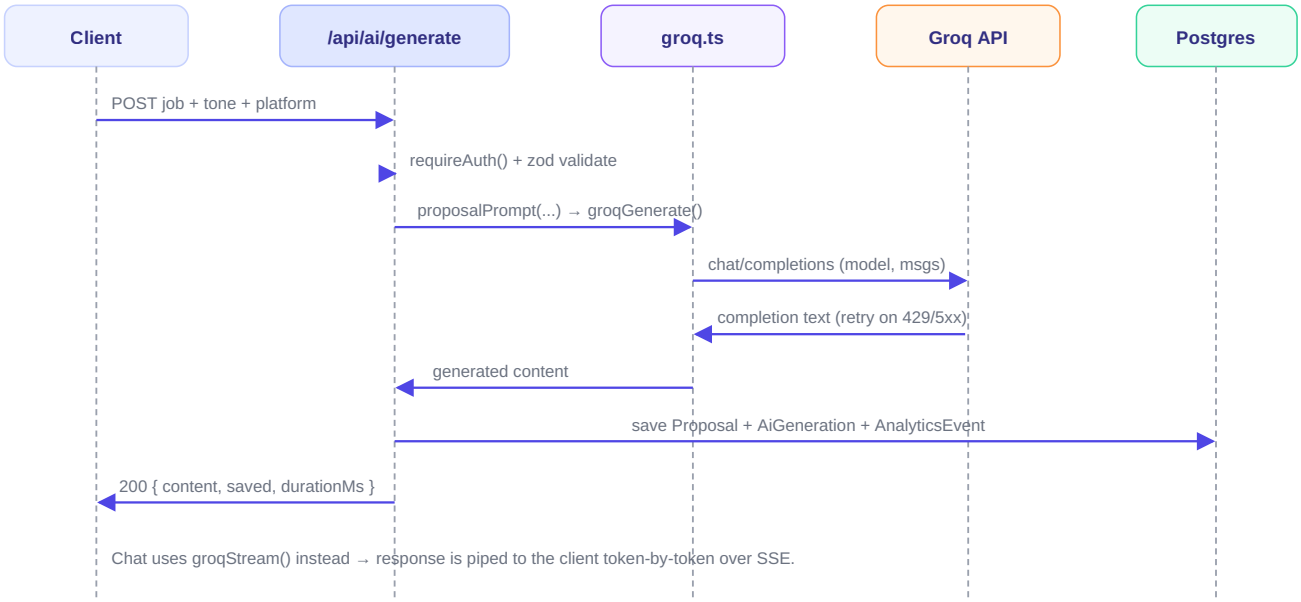


*Figure 2 — Sequence for a proposal generation. The streaming chat route swaps `groqGenerate()` for `groqStream()`.*

# 5. Data Model

The Prisma schema centres on `User`, which owns every generated artifact. All child relations cascade on delete, so removing a user cleanly removes their data.

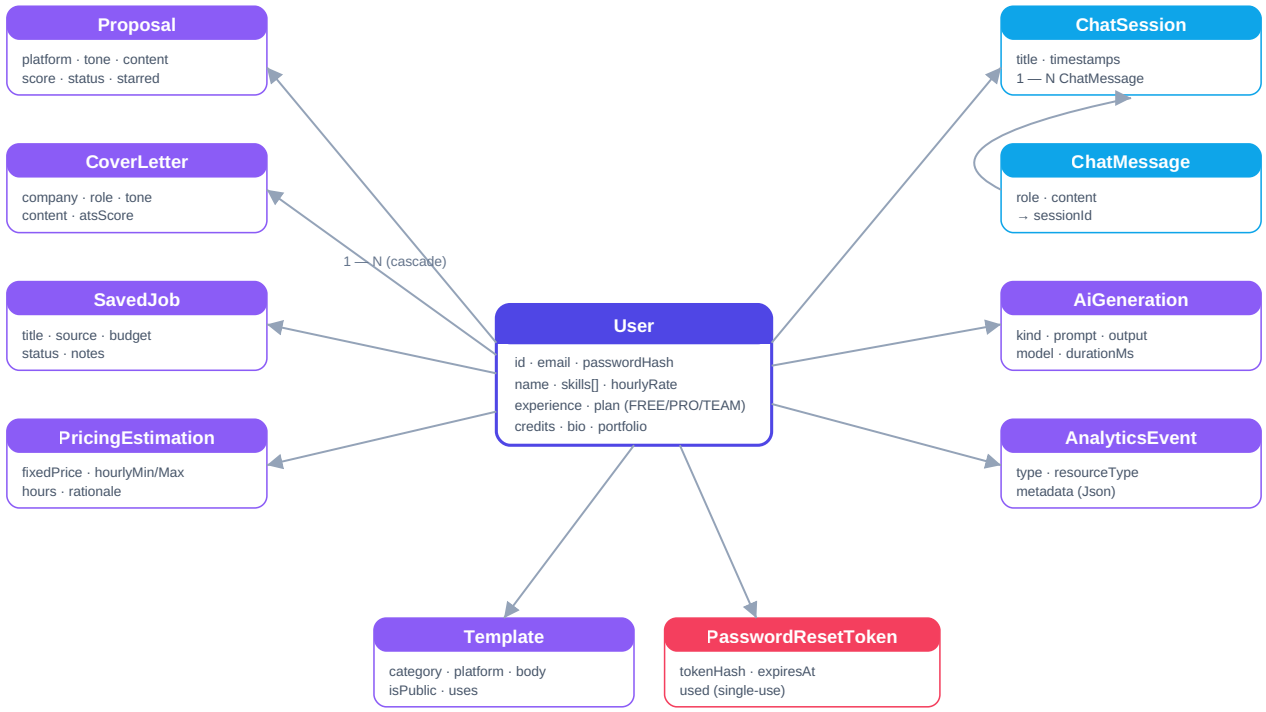


*Figure 3 — Entity relationships. User (1) owns many of every artifact; ChatSession owns many ChatMessage. All FKs cascade on delete.*

| Model | What it stores |
|---|---|
| `User` | Account, profile (skills, rate, experience, bio, portfolio), plan and credit balance. |
| `Proposal` | Generated proposal with platform, tone, score, pricing estimate, status (draft/sent/won/lost). |
| `CoverLetter` | Generated cover letter with company, role, tone, and ATS score. |
| `SavedJob` | Job leads with source, budget, notes, and a pipeline status. |
| `ChatSession` / `ChatMessage` | Persistent copilot conversations; messages cascade from their session. |
| `PricingEstimation` | Stored pricing/timeline outputs for a brief. |
| `AiGeneration` | Audit log of every AI call: kind, prompt, output, model, duration. |
| `AnalyticsEvent` | Product events (generated, exported, copied, won/lost) for the analytics page. |
| `Template` | Reusable proposal/outreach templates, optionally public. |
| `ExportedDocument` | Record of PDF/DOCX/TXT exports. |

| | |
|---|---|
| `PasswordResetToken` | SHA-256-hashed, single-use, 30-minute reset tokens. |

# 6. API Reference

All handlers live under `src/app/api`. Every authenticated route begins with `requireAuth()` and validates input with zod. Responses use the shared `ok()` / `fail()` envelope.

## AI

| Method | Endpoint | Description |
|---|---|---|
| POST | `/api/ai/generate` | Generate a proposal or cover letter; persists artifact + audit + analytics. |
| POST | `/api/ai/analyze` | Analyse a brief → strict JSON (category, complexity, pain points, red flags, tone). |
| POST | `/api/ai/score` | Score copy 0–100 with breakdown + fixes; can update a proposal's score. |
| POST | `/api/ai/pricing` | Estimate fixed price, hourly range, hours, days, rationale, risks. |
| POST | `/api/ai/rewrite` | Rewrite copy under an instruction (shorter, premium, punchier, ATS, …). |
| POST | `/api/chat` | Streaming copilot (SSE). Persists session + user/assistant messages. |
| GET | `/api/chat/sessions` | List chat sessions; `/[id]` reads or deletes one. |

## Auth

| Method | Endpoint | Description |
|---|---|---|
| POST | `/api/auth/signup` | Create account, hash password, issue session cookie. |
| POST | `/api/auth/login` | Verify credentials, set JWT cookie. |
| POST | `/api/auth/logout` | Clear the session cookie. |
| GET | `/api/auth/me` | Return the current user profile. |
| POST | `/api/auth/forgot-password` | Issue a hashed, single-use reset token. |
| POST | `/api/auth/reset-password` | Validate token and set a new password. |

## Workspace resources

| Method | Endpoint | Description |
|---|---|---|
| GET/ POST | `/api/proposals` | List / create proposals; `/[id]` reads, updates, deletes. |
| GET/ POST | `/api/cover-letters` | List / create cover letters; `/[id]` for single-item ops. |
| | `/api/saved-jobs` | Manage the job pipeline; `/[id]` for single-item ops. |

| | | |
|---|---|---|
| GET/ POST | | |
| GET/ POST | `/api/templates` | List and create reusable templates. |
| POST | `/api/analytics` | Record product analytics events. |

## 7. Authentication & Security

### Sessions

Stateless **HS256 JWT** signed with `jose`, carrying `sub`, `email`, and `plan`. Stored in an **HttpOnly**, `SameSite=Lax` cookie (`Secure` in production), 7-day expiry.

### Passwords

Hashed with **bcrypt** at cost 12. Plaintext is never stored or logged.

### Route protection

Middleware verifies the cookie before `/dashboard/*` renders; handlers independently call `requireAuth()` as defence-in-depth.

### Password reset

Tokens are **SHA-256 hashed at rest**, single-use, and expire in 30 minutes. In dev the reset link is returned for testing; in prod wire up an email provider.

**Input safety:** every route parses its body through a zod schema, so malformed or oversized input is rejected with a `400 validation_error` before any DB or AI work runs.

# 8. Project Structure & Deployment

## Directory layout

| Path | Contents |
|---|---|
| `src/app/(auth)` | Login, signup, forgot- and reset-password pages. |
| `src/app/dashboard` | Authenticated product: proposals, cover-letters, chat, analytics, saved-jobs, templates, settings. |
| `src/app/api` | All route handlers (AI, auth, workspace resources). |
| `src/components` | `landing/` marketing, `dashboard/` app surfaces, `ui/` Radix/shadcn primitives. |
| `src/lib` | `groq.ts`, `auth.ts`, `api.ts`, `prisma.ts`, `prompts.ts`, `pdf.ts`, `utils.ts`. |
| `prisma` | `schema.prisma` and `seed.ts`. |

## Configuration & run

Environment is defined in `.env` (template in `.env.example`): `DATABASE_URL`, `JWT_SECRET`, `JWT_EXPIRES_IN`, `GROQ_API_KEY`, `GROQ_MODEL`, and `NEXT_PUBLIC_APP_URL`.

| Command | Action |
|---|---|
| `npm run dev` | Start the Next.js dev server. |
| `npm run build` | `prisma generate` + `next build` (production bundle). |
| `npm run db:push` / `db:migrate` | Sync or migrate the Postgres schema. |
| `npm run db:seed` | Seed demo data via `tsx prisma/seed.ts`. |

A `Dockerfile` and `docker-compose.yml` are included for a containerised app-plus-Postgres setup; `GROQ_API_KEY` and `GROQ_MODEL` are passed through to the container environment.

> **Migration note (Gemini → Groq):** the AI client, all six AI routes, env files, README, docker-compose, UI labels, and the `AiGeneration.model` schema default were updated to Groq / `llama-3.1-8b-instant`. `src/lib/gemini.ts` was removed. A TypeScript type-check ( `tsc --noEmit` ) passes; a full build and a live Groq call should be run locally to confirm end-to-end behaviour.